# Project Plan Document: Product Management & Analytics Suite

Project Scope: ASP.NET Core MVC, Code-First MSSQL, Stored Procedures, Dual Caching, C# Delegates & MSTest

## 1. Understanding of the Requirements

The primary objective of this technical assessment is to design, develop, and test an enterprise-grade ASP.NET Core MVC web application demonstrating mid-level software engineering standards, clean architecture, SOLID principles, and high-performance data access.

### 1.1 Core Deliverables & Functional Scope

- **Product List Page:** Interactive product catalog with complete CRUD operations (Create, Read, Update, Delete) for managing: Product ID, Name, Category, Price, and Stock. Utilizes modal dialogs and asynchronous jQuery AJAX to eliminate disruptive page reloads.
- **Report Analytics Page:** Dedicated analytical dashboard providing business intelligence calculated directly on MSSQL:
- **- Calculation 1:** The average price of products in each category (No need to consider a separate category table). Executed via database stored procedure.
- **- Calculation 2:** The highest stock value category, computed as unit price multiplied by stock quantity (SUM(Price * Stock)). Executed via database stored procedure.

### 1.2 Architectural & Technical Constraints

- **Service & Repository Layer Pattern:** Decoupled into presentation, service, repository, and test projects. Strict isolation of database queries away from controllers and views.
- **Code-First Database Pattern:** Entity Framework Core with Code-First migrations targeting MSSQL (LocalDB / SQL Server Express). Tables and stored procedures are provisioned via migration scripts.
- **Multi-Tier Caching:** Service-layer performance caching supporting two distinct tiers: (a) 'Cached' for volatile data retained up to 5 minutes, and (b) 'CachedLong' for generic results retained indefinitely until evicted.
- **C# Delegates for Callbacks:** Implementation of C# delegates (ProductOperationCallback and StockAlertCallback) to handle asynchronous notification and audit workflows.
- **UI & Client Scripting:** Responsive layout styled with Bootstrap 5 and custom CSS, paired with jQuery for client-side DOM manipulation, modal handling, and live search filtering.
- **Automated Unit Testing:** Comprehensive test suite implemented using MSTest and Moq, isolating the service layer from physical database dependencies.

## 2. Implementation Strategy & Technical Plan

To strictly adhere to the requirement of separating concerns and avoiding direct database operations in web controllers, the solution adopts a Clean Layered Architecture divided into four specialized projects:

**1. ProductManagement.Core:** Contains domain entities (Product, Report projections), Data Transfer Objects (DTOs), the Result pattern, custom delegate definitions, and service interfaces (IProductRepository, IProductService, ICacheService). Has zero dependencies on EF Core or UI frameworks.

**2. ProductManagement.Data:** Manages database persistence via EF Core, ApplicationDbContext, Fluent API mappings, Stored Procedure execution, and Code-First migrations.
**3. ProductManagement.Web:** ASP.NET Core MVC presentation layer featuring controllers, ViewModels, Razor views, custom CSS, and jQuery AJAX handlers.
**4. ProductManagement.Tests:** MSTest test project containing automated unit tests with Moq to thoroughly verify service-layer CRUD logic, validation guards, and caching behavior.

### 2.1 Data Storage & Precision Rules

- Monetary values are mapped as decimal(18,2) to eliminate binary floating-point rounding errors.
- High-speed index configured on Category (IX_Products_Category) to optimize stored procedure GROUP BY operations.
- Automatic database migration execution on application startup to ensure zero-touch deployment on any developer machine.

### 2.2 Performance Caching & Invalidation Policy

A centralized ICacheService wraps IMemoryCache. To prevent stale cache data, every state-altering mutation in ProductService (Create, Update, Delete) automatically triggers cache eviction across both product listings and reporting calculation keys.

## 3. Identified Risks, Edge Cases & Mitigation Strategies

During architectural analysis, five critical engineering challenges were identified along with proactive mitigation strategies:

| Risk / Potential Issue | Impact | Architectural Mitigation Strategy |
|---|---|---|
| **Stale Cache Data** | Report or catalog displays outdated numbers after a product is created or edited. | Coordinated cache eviction in ProductService. InvalidateAllProductCaches() purges related cache keys immediately on any write mutation. |
| **Stored Procedure Sync Drift** | Colleagues running migrations don't receive the stored procedures. | Embed CREATE OR ALTER PROCEDURE SQL DDL directly inside EF Core Code-First migrations via migrationBuilder.Sql(). |
| **Floating Point Drift** | Rounding errors on currency calculations. | Enforce decimal(18,2) consistently across SQL schema, EF mappings, DTOs, and view formatting. |
| **Tampered Client Input** | Negative prices or stocks submitted past browser forms. | Dual-layer validation: DataAnnotations on ViewModels plus explicit domain validation guard clauses in ProductService. |
| **Environment Portability** | Connection failures when evaluated on different machines. | Configured for standard MSSQL LocalDB (Server=(localdb)\mssqllocaldb;Database=ProductDb) with automated startup migration. |

# 4. Pseudocode Covering Required Logic

## 4.1 Dual Caching Logic (Cached vs CachedLong)

```
CLASS MemoryCacheService IMPLEMENTS ICacheService:
    FUNCTION Cached<T>(key, factory):
        RETURN GetOrCreate(key, factory, Expiration = 5 Minutes)

    FUNCTION CachedLong<T>(key, factory):
        RETURN GetOrCreate(key, factory, Expiration = NULL) // Retained indefinitely

    FUNCTION GetOrCreate<T>(key, factory, expiration):
        IF cache.TryGetValue(key, out existingValue) THEN:
            RETURN existingValue
        value = factory()
        IF expiration HAS VALUE THEN:
            cache.Set(key, value, AbsoluteExpirationRelativeToNow = expiration)
        ELSE:
            cache.Set(key, value) // No expiration until explicitly evicted
        RETURN value
```

## 4.2 Service Layer CRUD with Invalidation & Delegate Callbacks

```
DELEGATE ProductOperationCallback(operationName, product, isSuccess, message)
DELEGATE StockAlertCallback(productId, productName, stockCount)

FUNCTION CreateProductAsync(dto, callbackDelegate, stockAlertDelegate):
    // 1. Validation Guard
    IF dto.Price <= 0 OR dto.Stock < 0 OR string.IsNullOrEmpty(dto.Name) THEN:
        callbackDelegate?.Invoke("Create", NULL, FALSE, "Validation failed.")
        RETURN Result.Failure("Invalid product parameters.")

    // 2. Persist Entity via Repository
    product = MapToEntity(dto)
    savedProduct = repository.AddAsync(product)

    // 3. Coordinated Cache Eviction
    cacheService.Remove("products:all")
    cacheService.Remove("reports:summary:5min")
    cacheService.Remove("reports:summary:long")

    // 4. Delegate Callbacks
    IF savedProduct.Stock < 10 THEN:
        stockAlertDelegate?.Invoke(savedProduct.Id, savedProduct.Name, savedProduct.Stock)

    productDto = MapToDto(savedProduct)
    callbackDelegate?.Invoke("Create", productDto, TRUE, "Product created successfully.")
    RETURN Result.Success(productDto)
```

## 4.3 Stored Procedure SQL Definitions

```
-- Calculation 1: Average Price Per Category
CREATE OR ALTER PROCEDURE sp_GetAveragePricePerCategory AS
BEGIN
    SET NOCOUNT ON;
    SELECT Category, CAST(AVG(Price) AS DECIMAL(18,2)) AS AveragePrice, COUNT(Id) AS
```

```
ProductCount
    FROM Products
    GROUP BY Category
    ORDER BY Category ASC;
END;

-- Calculation 2: Highest Stock Value Category
CREATE OR ALTER PROCEDURE sp_GetHighestStockValueCategory AS
BEGIN
    SET NOCOUNT ON;
    SELECT TOP 1 Category, CAST(SUM(Price * Stock) AS DECIMAL(18,2)) AS TotalStockValue,
SUM(Stock) AS TotalStockQuantity
    FROM Products
    GROUP BY Category
    ORDER BY TotalStockValue DESC;
END;
```

## 5. Development Verification & Quality Sign-Off

All components outlined in this project plan were developed, integrated, and verified:

- [COMPLETED] Multi-project Clean Architecture solution created targeting .NET 8 (LTS).
- [COMPLETED] Domain entities, validated DTOs, and keyless projection models implemented.
- [COMPLETED] EF Core DbContext configured with Fluent API and MSSQL LocalDB connection.
- [COMPLETED] Stored procedures embedded in migrations and verified on SQL Server.
- [COMPLETED] MemoryCacheService implementing Cached (5 min) and CachedLong with cache eviction.
- [COMPLETED] Custom C# delegates (ProductOperationCallback and StockAlertCallback) implemented.
- [COMPLETED] Responsive MVC views (Product List and Report) built with Bootstrap 5 and jQuery AJAX.
- [COMPLETED] Full MSTest unit test suite with Moq: 15 passing tests, 0 warnings, 0 errors.